

Assignment 10

Page No.

Date: / /

1] Explain concept of Structure? explain with example.

- It is a user-defined datatype.
- Holds - Heterogeneous datatype based on programmer's need.
- Concept of Structure extend in C++ & Java known as : 'Class'
- Can store any primitive, derived & user defined data in it.

Ex: struct Hello

```
{ int N;
```

```
  char ch;
```

```
}
```

```
int main()
```

```
{ struct Hello obj;
```

```
  obj.N = 10;
```

```
  obj.ch = 'a';
```

```
}
```

2] Explain concept of padding in memory allocation of structure.

include <stdio.h>

```
{ int i;
```

```
  char ch;
```

```
  float f;
```

```
  double d;
```

```
}
```

```
int main()
```

```
{ struct Demo obj;
```

```
  printf("size of Demo structure is = %d\n", sizeof(obj));
```

```
  return 0;
```

```
}
```

Here Actual memory = 17 bytes
But shows memory = 24 bytes

- 1) In above program, Demo Struct is 17 bytes but when we execute it, it gives us 24 bytes, in that 7 bytes are considered padding.
 - 2) Padding is considered as allocated memory inside the object that we can't access.
 - 3) It is considered as wastage of memory.
 - 4) To Access the elements in fastest way, Compiler will assign some extra memory i.e. 'Padding'.
- E.g. $\text{int } a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z$
- 5) Largest element is of type double i.e. 8 bytes due to which memory for each element in our structure get allocated in terms of 8 bytes irrespective of their original sizes.

	<u>Structure</u>	<u>Union</u>
Keyword	struct	union
Size	Size = Total size of all its members	Size = size of largest member
Storage	Store data for multiple members at same time	Data for only 1 variable at a time
Memory Allocation	Also separate memory allocation	Allocation only for the - 'largest member in size'

- 4) Which type of data can we store in Struct? Explain with a program.
- We can store heterogeneous type of data in a structure, as per the requirement.


```

#include <stdio.h>
struct Demo
{
    int i;           // integer
    float *p;        // pointer
    double d;        // double
};

int main()
{
    struct Demo obj;
    int i = 100;
    float f = 11.11f;
    obj.p = &f;
    obj.i = 100;
    obj.d = 333.3333;

    printf("%d\n", *obj.p);
    printf("%f\n", *obj.p);
    printf("%f\n", *obj.d);
    return 0;
}

```

5] What are different ways to initialize members of structure?

→ 1] We can initialize object of structure by using 'member initialization list':

struct Demo obj1 = { 200, 100, 333.3333 };

2] We can initialize with designated initialization list

Ex: struct Demo obj2 = { p = 200; i = 100; d = 333.3333 };

6] Why we can't initialize members of structure at time of declaration.

→ At time of declaratⁿ of structure, we can't initialize its member with its value because, initialization is not allowed at that point there is no memory allocation for the structure.

Eg. Struct Demo

```
{ int i = 10; } // not allowed.
```

7] Write the difference between Structure and Array.

	Structure	Array
data type	User-defined datatype	Derived Datatype
Holds	Heterogeneous datatype members	Homogenous type of members.
Access memory allocation	By using $\cdot$ / $\rightarrow$ operator Non-consecutive (use of padding)	Using $[]$ operator. Consecutive memory allocation.
Usage	Useful for grouping logically related variables.	Useful for storing and manipulating similar data collection.
Initializ ⁿ	Can't be initialized at time of declaration	May / may not be initialized at time of declaration.
Nested usage	Can contain arrays & other structures as members.	Cannot contain structures as elements (only primitive type)
E.g.	Struct Demo <pre>{ int i; float *p; }</pre>	<pre>int Arr[5] = {10, 20, 30}; or int Arr[5];</pre>

8] What is the difference between (.) & Indirect access (->) operator, explain with example & program.

→ A] Direct Accessing Operator (.)
When we have object of structure or Union then, we can access its value directly using dot (.) operator.

E.g. Direct $\rightarrow$ $\text{Box} \rightarrow \text{Box} \rightarrow \text{Box}$ $\rightarrow$ Box $\rightarrow$ Box $\rightarrow$ Box

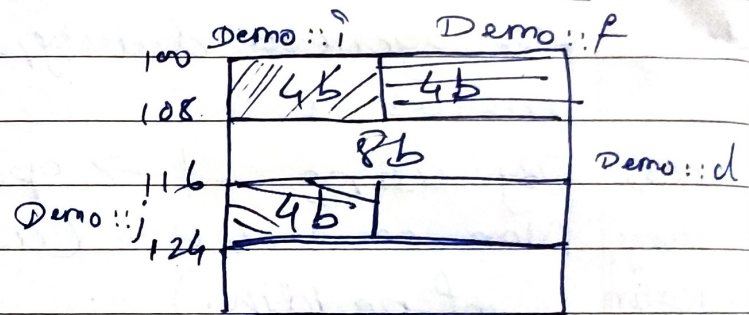
B] Indirect Accessing Operator (->)

When we have a pointer, which points to object of structure/union, then to access its members we use arrow (->) operator.

E.g. Table $\rightarrow$ $\text{Table} \rightarrow \text{Table} \rightarrow \text{Table}$ $\rightarrow$ Table $\rightarrow$ Table $\rightarrow$ Table

9] Draw memory layout for below structure

```
struct Demo
{
    int i;
    float f;
    double d;
    int j;
};
```



$\text{Diagonal lines} = \text{int } i \text{ (4 bytes)}$

$\text{Horizontal lines} = \text{int } j \text{ (4 bytes)}$

$\text{Float } f = \text{Horizontal lines} \text{ 4 bytes}$

Total memory - 20 bytes.

Actually needed = $4 + 4 + 8 + 4 = 20$ bytes.

10] Find problem & correct it & draw memory layout
struct Demo

```
{
    int i;
    float f = 10.5;
    double d;
};
```

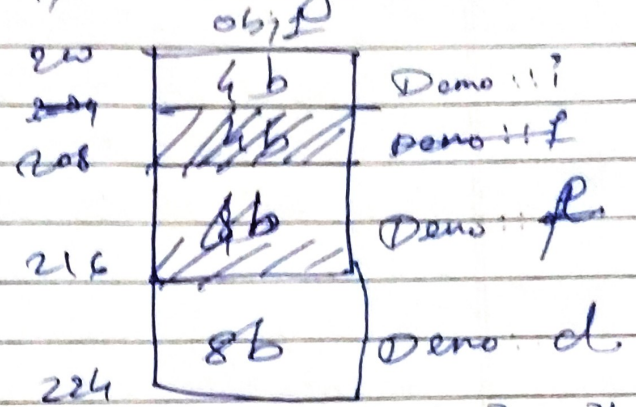

Initialization of member at time of declared is not allowed.

∴ float f = 10.5; // X wrong

```
struct Demo
{
    int i;
    float f;
    double d;
};
```

int main()

```
{
    struct Demo obj1 = {0, 10.5, 0.0};
    return 0;
}
```



Q.9 Diagram

